

Handling Side Effects & Working with the `useEffect()` Hook

A beginner-friendly guide to React side effects

1. What is a Side Effect?

A **side effect** is anything your component does that goes **outside of just showing things on the screen**. React's main job is to display UI — but sometimes your app needs to do other things too.

Common examples of side effects:

- Fetching data from an API (like getting a list of users)
- Setting a timer with `setTimeout` or `setInterval`
- Directly changing the browser title or URL
- Reading/writing to `localStorage`
- Subscribing to events (like window resize or keyboard press)

Simple Analogy: Think of a component as a chef whose job is to plate a dish (render UI). A side effect is when the chef also calls the supplier, checks inventory, or turns on the oven — things outside of just plating.

2. The Infinite Loop Problem

If you run a side effect **inside the component body directly** (not inside `useEffect`), it can cause an **infinite loop**. Here's why:

Step 1: Component renders → side effect runs (e.g., updates state)

Step 2: State update → component re-renders

Step 3: Re-render → side effect runs again → state updates again

Step 4: This goes on forever! ■

Fix: Wrap the side effect inside `useEffect()` so React controls when it runs.

3. Using useEffect for Side Effects

useEffect() is a React Hook that lets you run code **after** the component renders. It takes two things: a function (your side effect) and a dependency array.

Basic structure:

```
useEffect(() => {  
  // Your side effect code here  
  fetch('https://api.example.com/data')  
  .then(res => res.json())  
  .then(data => setData(data));  
}, [dependency]); // runs when 'dependency' changes
```

4. Not All Side Effects Need useEffect

■ Use useEffect for:

- Fetching data from API
- Setting up subscriptions
- Starting a timer
- Syncing with external system
- Reading from DOM after render

■ Don't use useEffect for:

- Computing a value from state/props
- Filtering or transforming a list
- Simple event handlers
- Updating state based on another state (use the updater function instead)

5. Syncing with Browser APIs

useEffect is great for **updating browser-level things** like the page title, window size, or scroll position — things that live outside React.

Example — Update browser tab title:

```
useEffect(() => {  
  document.title = `You have ${count} messages`;   
}, [count]); // Updates title whenever count changes
```

6. Understanding Effect Dependencies

Dependency Array	When does the effect run?
No array (missing)	After every render

[] (empty array)	Only once — when component first appears
[value]	Once on mount + whenever 'value' changes
[a, b]	Whenever 'a' or 'b' changes

7. The Cleanup Function

When a component is **removed from the screen** (unmounted), you often need to **clean up** what your effect started — like clearing a timer or cancelling a subscription. You do this by **returning a function** from `useEffect`.

Example — Cleanup a timer:

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log('Tick!');
  }, 1000);

  return () => {
    clearInterval(timer); // Cleanup when component unmounts
  };
}, []);
```

***Think of it like this:** You open a door (start effect) and the cleanup closes the door before you leave the room (unmount).*

8. Problem with Object & Function Dependencies

In JavaScript, **objects and functions are recreated** every time a component renders — even if their content looks the same. React sees them as **new/changed**, which can cause your `useEffect` to run more than expected.

Example of the problem:

```
// This object is NEW every render, even if content is same
const options = { theme: 'dark' };

useEffect(() => {
  applySettings(options);
}, [options]); // ■■ Runs every render!
```

Solution: Move the object/function inside the `useEffect`, or use **`useMemo`** / **`useCallback`** to keep stable references across renders.

9. The `useCallback` Hook

`useCallback` is a Hook that **remembers a function** between renders. It only creates a new function when its dependencies change. This prevents unnecessary re-runs of `useEffect` when a function is a dependency.

Usage:

```
const fetchData = useCallback(() => {  
  fetch(`/api/data?id=${userId}`)  
    .then(res => res.json())  
    .then(data => setData(data));  
}, [userId]); // Only recreated when userId changes  
  
useEffect(() => {  
  fetchData(); // Safe to use as dependency  
}, [fetchData]);
```

When to use `useCallback`: When you pass a function as a prop to a child component, or when a function is listed as a dependency in `useEffect`.

10. Optimizing State Updates with `useEffect`

You can use `useEffect` smartly to **avoid unnecessary work** and keep your app fast. A few key optimization tips:

- **Always provide a dependency array:** Avoid running the effect on every render unless needed.
- **Debounce fast-changing values:** If a user types quickly, wait until they stop before running the effect.
- **Cancel async operations:** If the component unmounts mid-fetch, use a cleanup flag to ignore stale responses.
- **Avoid setting state in a loop:** Batch state updates together to prevent multiple re-renders.

Cancelling stale fetch example:

```
useEffect(() => {
```

```
let isCancelled = false;

fetch('/api/data')
  .then(res => res.json())
  .then(data => {
    if (!isCancelled) setData(data); // Only update if still mounted
  });

return () => { isCancelled = true; }; // Cleanup
}, []);
```

Quick Summary

<code>useEffect(fn, [])</code>	Run once when component loads
<code>useEffect(fn, [x])</code>	Run when x changes
Return a function	Cleanup (e.g., clear timers)
<code>useCallback</code>	Keep function reference stable
Avoid objects in deps	Move inside effect or memoize